

Your 10x Solution

The capstone for interns with their own idea. You saw a problem. You have a solution. Build it for real — your problem, your technology, your design. This brief is only the frame.

Your own idea

At least 5 program concepts

Difficulty: you set it

No presentation

\$0 · no credit card

WHO THIS IS FOR

Interns with their own idea, solution, or personal project

YOU WILL PROVE

You can turn an idea into a scoped, running system built from program concepts

SUBMISSION (ON THE PORTAL)

Public GitHub repo link + the overview document "My 10x Solution - {Your Name and Surname}"

How to read this document: it is short on purpose. Read sections 1 and 2 now — that takes 5 minutes. Come back to the rest when you need it. **Bold text marks the decisions you must make.** Every checkbox is something you can finish and tick. One section = one idea.

Contents

1. Is this capstone for you?	1	6. Submission — no presentation.....	4
2. The concept rule	2	7. Realistic scope.....	4
3. Constraints	2	8. Steal from your own assignments.....	4
4. Build order — five milestones	3	9. Glossary	6
5. Definition of done	3		

1 · Is this capstone for you?

This capstone is for interns who have their **own idea**. You saw a problem. You have a solution for it. Maybe you already started to build it. This is your invitation to build it for real — and make it count as your capstone. We really encourage you to work on your own solution: your own project gives the most meaningful work, and it becomes the strongest item in your portfolio. You define the problem, the technology, and the design. **It is your canvas.**

Answer one question: **do you have your own idea?**

- **Yes** — a problem you saw, a tool you wish existed, a project you already dream about → this capstone is for you. Build that.
- **No, and that is fine** — you want a defined brief with a known shape → pick one of the standard capstones from the portal menu instead. They are equally respected.

One rule stays the same for everyone: you complete **one** capstone.

2 · The concept rule — the heart of this brief

Your solution is free-form. The engineering is not. Your build must show that you absorbed the program.

Pick at least 5 of these 7 concepts and implement them:

#	Concept	What counts
1	API endpoints	A real HTTP API with correct status codes and validation
2	Database	Real persistence — your data survives a restart
3	Authentication	Users log in; protected routes are actually protected
4	Background jobs or cron jobs	Slow work off the request path, or work on a schedule
5	Reporting — PDF or email	Your system produces a report as a PDF, or sends one by email
6	Caching logic	A result that is expensive to compute is stored and reused
7	LLM integration	One narrow AI job behind an endpoint, with validation and a cost log

The swap rule. Some good solutions do not need all 7 concepts. That is normal. If a concept does not fit your solution, **swap** it for one of these:

Swap in	What counts
Web scraping pipeline	Polite collection of external data, validated and idempotent
Webhook receiver	Signed, verified, safely-retried incoming events
RAG with citations	Answers grounded in your own data, with sources
Agent with guardrails	An AI that acts, inside strict limits
Containerized stack	Your whole system starts with <code>docker compose up</code>
Test suite	The scary cases covered, deterministic, runnable in one command
Rate limiting / quotas	Limits enforced at the boundary with honest status codes
Deployment	Your system runs on a free hosting tier, reachable by URL

THREE RULES — 1. At least 5 concepts in total. Count them. 2. Maximum 2 swaps — at least 3 of your 5 come from the first table. 3. Write your list down: your overview document and your README name the 5+ concepts, with one line per swap that says why the original concept did not fit.

3 · Constraints — read before you build

- **\$0 stack, no credit card.** Free tools, free tiers, local services, test modes. Same rule as everything in this program.
- **No secrets in Git.** Keys and passwords live in environment variables only. Never commit them. Never log them.
- **No other people's personal data.** Use your own data, seeded data, or public data.
- **Your repo, your code.** AI help is allowed and encouraged — but you must understand every line. You own it.
- **The scope guard.** Your core must be buildable in about 3 weeks of part-time work. Write your core as a feature list. **If the list has more than 5 features, cut it down.** A small solution that runs beats a big solution that does not. This is the rule that saves your capstone.

4 · Build order — five milestones, each with a timebox

Do the milestones in order. Finish one before you start the next. The timeboxes are honest estimates, not deadlines.

M1 The one-pager

one evening

Decide before you build. One page protects three weeks.

Write one page: the problem (3 sentences) · who has this problem · your **10x claim** — what becomes ten times faster, cheaper, or easier · your **5+ concepts** from section 2 · one explicit **non-goal** (a thing you will NOT build). This page later becomes your submission document, so nothing here is wasted.

M2 The walking skeleton

one weekend

The thinnest slice that runs, end to end.

One API endpoint → logic → database → response. Ugly is fine. It must run. Everything after this is adding muscle to a skeleton that already walks.

M3 The concepts, one at a time

about two weeks

One open task at a time. Always.

Add your remaining concepts **one at a time**. Finish one — it runs, it is committed — before you start the next. Never have two half-built concepts open at once. This is the same rhythm as the assignments: small stage, test, commit.

M4 Make it runnable by a stranger

one weekend

Your work only counts when someone else can run it.

A README with exact run steps · a seed script or demo data · a 5-minute demo path written down ("open X, click Y, see Z"). Test it: could someone who has never seen your project run it from the README alone?

M5 Package and submit

one evening

The last mile is short. Walk it.

Finish the overview document (section 6) · walk the Definition of done (section 5) and tick every box · submit.

5 · Definition of done (core)

Done = every box ticked. Each one is checkable in minutes.

- ☐ The **problem is written down**: your overview document exists, named as section 6 says, and a stranger can understand the problem from it.
- ☐ **At least 5 concepts** from section 2 are implemented — listed in the README as a small table: *concept* → *where it lives in the code*.
- ☐ **Maximum 2 swaps**, each with a one-line reason in the overview document.
- ☐ The system **starts with one or two documented commands** on a clean machine.

- ☐ **Demo data is seeded** and the README's 5-minute demo path works.
- ☐ No secrets in the repo; `.gitignore` covers env files, databases, and generated artifacts.
- ☐ The repo is **public on GitHub**.

6 · Submission — two items, no presentation

There is **no live presentation** for capstones. There are too many interns for live sessions. You do not present your work. Your repository and your document speak for you — so make them speak clearly.

When you submit this capstone **on the portal**, submit two items:

1. **The link to your public GitHub repository.** If the repository holds other work too, put the capstone in its own clearly named folder.
2. **One overview document**, as `.md` or `.docx`. You must name the file exactly like this:

My 10x Solution - {Your Name and Surname}

Example: `My 10x Solution - Ana Kovac.md`. Upload this file on the portal together with your repository link. (The capstone is called *Your 10x Solution*. Your document says *My* — because by then, it is.)

The overview document answers two questions, in one to two pages: **1. What is the problem you are solving?** Describe the problem so a stranger understands it. **2. How did you implement your solution?** Explain how it works in plain words, name the 5+ concepts you implemented (with one line per swap), and give the steps to run it.

7 · Realistic scope — what NOT to build

- No mobile apps. No pixel-perfect UI. A plain page or a clean API is enough.
- No real money, no real user base, no scale. Test modes and seeded data are correct here.
- No second product. One problem, one solution. Your non-goal from M1 protects you — when a shiny idea appears mid-build, write it in the README under "Future ideas" and get back to the milestone.

8 · Steal from your own assignments

You are not starting from zero. You already built every part once:

Your solution needs	You already built it in
Login and protected routes	A4 — Auth: login & protect
A database with real data	A2 · A3 — CRUD to database, containerized stack
Slow work in the background, or on a schedule	A7 — Your first background job
A PDF report or an email	A8 — PDF report generator
An AI feature	A17 — Put an LLM behind your API
External data	A9 · A16 — the scraping assignments

Copying from your own assignment repos is not cheating. It is the point. The capstone is **assembly, not a greenfield panic**.

Stretch (optional)

- ☐ **Deploy it** — a free hosting tier, reachable by URL. A live link in a portfolio hits different.

- ☐ **Record a 2-minute demo video or GIF** and link it in the README — it does the presenting you don't have to do.
- ☐ **Measure the 10x** — one number, before vs after ("finding X took 20 minutes; now it takes 30 seconds").
- ☐ **A test suite** for the scary cases (if it wasn't one of your 5).

9 · Glossary

Plain-language definitions. No definition depends on another — read them in any order.

Word	What it means
Concept	One building block from the program — auth, database, jobs, reporting, caching, LLM integration. Section 2 lists them all.
Swap	Replacing a concept that does not fit your solution with one from the second table in section 2. Maximum 2.
10x claim	One sentence that says what your solution makes ten times faster, cheaper, or easier. It keeps your build pointed at the problem.
Non-goal	A thing you decide NOT to build, written down. The cheapest scope protection that exists.
Walking skeleton	The thinnest end-to-end slice that runs: one request goes in, touches every layer, comes back. You grow it; you never rewrite it.
Timebox	A time budget for a milestone. When the time is spent, you stop and ship what you have.
Seed / demo data	Data inserted by a script so a stranger can see your solution work without creating anything by hand.
Caching	Storing an expensive result so the next request reuses it instead of recomputing it.
Cron job	A task that runs at fixed times on a schedule — every day at 08:00 — with no request involved.
LLM	A large language model — the AI behind assistants like Claude or ChatGPT. In this program you put one narrow LLM job behind an API.
Scope	Everything you plan to build. The scope guard in section 3 keeps it small enough to finish.
README	The front page of a repo: what this is, how to run it, how to use it.

FlyRank Internship · Backend Development Track · Capstone — Your 10x Solution. All tools required are free with no credit card. No live presentation — submission is a public GitHub repository link plus one overview document.